DataGatherer

Technical assessment — status, risks and next steps — October 2026

Victor Gabriele De Matteis • github.com/ViDem02/DataGatherer

1 Summary

What it is. A web application that turns screenshots of Instagram posts into a spreadsheet of who used which hashtag, how many times. An operator crops the account name and the hashtags on each screenshot, text recognition (OCR) reads them, the operator confirms or corrects the text, and the application builds a user–hashtag matrix ready for network or market analysis.

Status. Suitable today as a tool run on one computer by one person or a small team. It is not yet suitable to be offered to customers or hosted on a network: the public code history still holds old secrets and third-party personal data, the collected data has no GDPR framework, and there is no HTTPS, automated backup or automated test gate.

Recommendation. Use it internally as is. Before any wider use, complete the four items of Section 6 marked "before any external use"; together they are estimated at one to two weeks.

Area	Rating	Basis
Core functionality	**Ready**	Upload, cropping, OCR, verification, matrix export all work end to end.
Ease of installation	**Ready**	Docker Desktop plus a double-click; ready-made images (about 780 MB; 36 seconds to download in our test).
Application security	**Ready**	Reviewed and hardened in October 2026: per-user data isolation, secure login, login lockout, private files.
Repository hygiene	**Not ready**	Old secrets and personal data remain in the public git history.
Data protection (GDPR)	**Not ready**	Personal data of third parties is collected without legal basis, retention or deletion process.
Reliability and operations	**Partial**	No automated backups; long batch computations stop if the server restarts.
Quality assurance	**Partial**	39 automated tests, run by hand; CI builds the images but does not run the tests yet.
Scalability	**Partial**	One server; fine for thousands of images, not for many simultaneous teams.

Table 1: Readiness at a glance.

2 Value

The repository's own measurements (`stats.md`) report 1,089 screenshots processed. They estimate about 2 seconds of automated work per image against about 30 seconds for manual transcription, i.e. roughly 93 % less effort. These figures are estimates, not timed measurements, and they exclude the operator's verification of each text, which remains necessary because OCR misreads accents and similar letters.

The output is a standard CSV file (rows: hashtags, columns: accounts, cells: number of posts), directly usable in spreadsheets, Python/R or network-analysis tools.

3 How It Is Built

The application is reachable only from the computer it runs on, by design. Each user account sees only its own projects and images; an administrator manages accounts through a built-in admin panel.

Component	Technology
User interface	React web app, served by nginx; works in any modern browser.
Application	Python, Django 5.2 LTS (supported until April 2028), REST API, Tesseract OCR.
Data	PostgreSQL 16; uploaded images on a Docker volume.
Delivery	Three Docker containers; images built automatically for Intel/AMD and ARM and published on GitHub.
Runs on	Windows, macOS or Linux with Docker Desktop.

Table 2: Technology.

4 What Was Done in October 2026

A review on 12 October 2026 found the application unsafe to expose: anyone could download the whole database, users could read and change each other's data, and secrets were stored in the code. The same day:

- **Security:** every hole found was closed; login tokens moved to browser-protected cookies; repeated wrong passwords lock the account for 15 minutes; uploaded images became private; dependency scans now report no known vulnerabilities (they reported 64 in the backend and 116 in the frontend).
- **Installation:** from a multi-step technical setup (Node.js, secrets written by hand, 10–20 minutes of building) to a double-click that generates its own secrets and downloads ready-made images.
- **Correctness:** batch jobs no longer report success when part of the work failed; the matrix export went from one database query per cell to three queries in total (a 20×20 example: 442 queries before); edits that fail can no longer delete existing data.

Each fix was verified with automated tests or end-to-end checks, including a real browser.

5 Risks

Risk	Level	Mitigation
Old secrets, database dumps and a third party's utility bill in the public code history	High	Rotate every secret (done locally), then rewrite the history or recreate the repository.
Personal data of Instagram users (GDPR); platform terms of use	High	Legal review: legal basis, retention period, deletion on request, record of processing; check Instagram's terms.
Data loss	Medium	Scheduled database and image backups with a tested restore.
Wrong numbers in the matrix from operator errors (duplicate screenshots, OCR typos accepted)	Medium	Guards in the interface: duplicate detection, warnings before saving (designed, not built).
Regressions from future changes	Medium	Run the tests automatically on every change.
Exposure if run on a network	Medium	Keep it local, or add HTTPS and secure cookies before exposing it.
Windows installation not yet tried on a real PC	Low	One test run on Windows.

Table 3: Main risks.

6 Next Steps

Action	Effort	When
Purge the code history and rotate any remaining secret	1 day	Before any external use
GDPR assessment of the collected data	legal, 1–2 weeks	Before any external use
Automated tests on every change, scan for committed secrets	2–3 days	Before any external use
Automated backups and a tested restore	1–2 days	Before any external use
Operator guards against duplicate screenshots and OCR typos	2–3 weeks	To improve data quality
HTTPS, background job queue, monitoring, staging environment	2–4 weeks	Only for a hosted, multi-team service

Table 4: Recommended actions. Efforts are estimates for one developer.

Detailed technical documentation is available on request: a change record of the security work, the user and maintenance workflows, and an in-depth description of the architecture and algorithms.